

1. 문서 개요

사용자가 제공한 강의 슬라이드를 바탕으로 딥러닝 아키텍처의 발전 방향을 한 흐름으로 연결해 설명한다. 핵심 관점은 “정보의 보존(preservation)”이다. 깊은 네트워크에서는 기울기가 사라지고, 객체 검출에서는 다양한 크기의 물체를 처리하는 과정에서 계산 병목이 생기며, 이미지 분할에서는 풀링 때문에 공간 해상도와 위치 정보가 사라진다. 각 문제를 어떤 구조적 장치가 해결했는지와, 그것이 PyTorch 코드에서 어떻게 구현되는지를 함께 보여 주는 것을 목표로 한다.

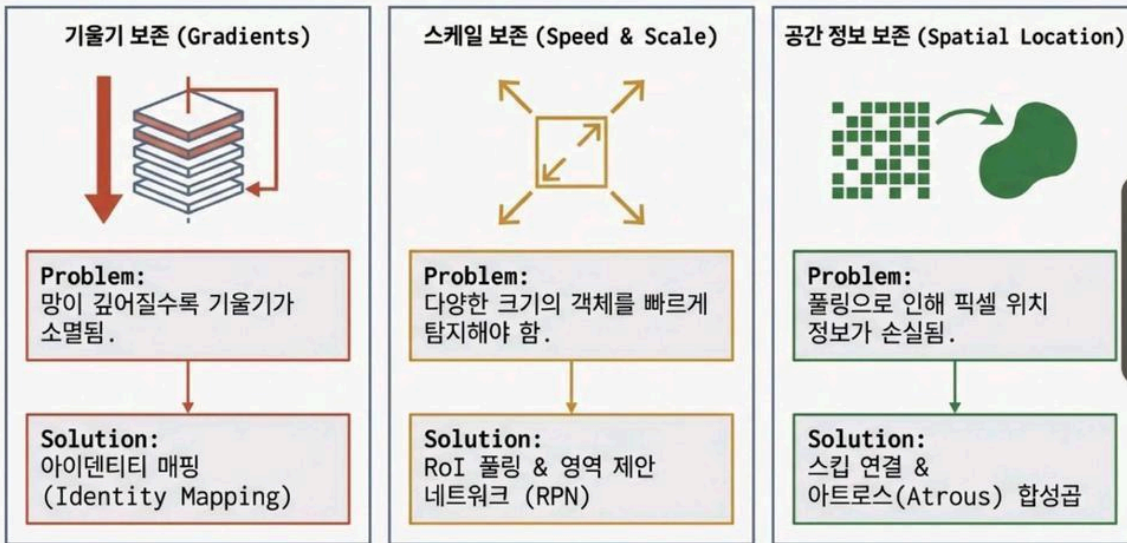
따라서 단순한 모델 소개가 아니라, 모델이 왜 등장했는지, 어떤 병목을 겨냥했는지, 구조가 어떤 수학적/공학적 의미를 갖는지, 그리고 구현 시 **forward** 흐름을 어떻게 읽어야 하는지까지 단계적으로 정리한다.

2. 딥러닝 아키텍처를 관통하는 핵심 주제: 신호의 보존

딥러닝 아키텍처의 진화를 세 가지 축으로 요약한다. 첫째는 깊은 네트워크 학습에서의 기울기 보존, 둘째는 객체 검출에서의 스케일 및 속도 보존, 셋째는 분할 문제에서의 공간 위치 정보 보존이다. 이 세 축은 서로 다른 과제처럼 보이지만, 본질적으로는 “중요한 정보를 잃지 않고 뒤 단계까지 전달하는 방법”이라는 공통 질문으로 연결된다.

- 기울기 보존: 깊은 망일수록 역전파 신호가 약해지는 문제를 **residual/identity shortcut**으로 완화한다.
- 스케일 보존: 다양한 크기의 객체를 한 네트워크 안에서 빠르게 처리하기 위해 **RoI Pooling**과 **RPN**을 도입한다.
- 공간 정보 보존: 풀링과 **stride**로 약해진 위치 해상도를 **skip connection**, **atrous convolution**, **upsampling**으로 복구한다.

핵심 과제: 신호의 보존 (Preserving the Signal)



신호 보존 관점의 세 가지 핵심 과제

[종합] 심층 신경망 설계의 본질: 정보의 보존 (Preservation)

	목적 (Goal)	한계 (Bottleneck)	아키텍처적 해결책 (Architectural Solution)
ResNet	깊은 망의 학습 (Gradients)	깊어질수록 기울기가 기울기가 소멸됨.	아이덴티티 매핑 (Identity Shortcut) - 잔차(Residual)를 더해 기울기 고속도로 구축.
Faster R-CNN	빠른 다중 객체 검출 (Scale & Speed)	가변적 객체 크기와 느린 외부 알고리즘.	RPN & RoI 풀링 - 네트워크 내부에서 고정 크기 출력 및 후보 영역 즉시 제안.
U-Net & DeepLab	픽셀 단위 위치 파악 (Spatial Info)	풀링으로 인한 공간 해상도 파괴.	스킵 연결 & Atrous 합성곱 - 특성 맵 직결 및 수용 영역 확장을 통한 위치 해상도 보존.

ResNet / Faster R-CNN / U-Net & DeepLab을 하나의 보존 관점으로 묶은 요약표

3. ResNet: 깊은 네트워크 학습을 가능하게 만든 잔차 학습

ResNet이 등장한 배경은 “층을 더 쌓으면 무조건 성능이 좋아질 것”이라는 직관이 실제로는 항상 맞지 않았기 때문이다. 슬라이드에서는 20층 모델보다 56층 모델이 훈련/테스트 오차에서 오히려 불리해질 수

있음을 보여 준다. 이 현상은 단순히 과적합이라기보다, 더 깊은 네트워크를 최적화하는 과정 자체가 어려워지는 **degradation problem**으로 이해해야 한다.

ResNet은 원하는 함수 $H(x)$ 를 직접 학습하는 대신, $F(x)=H(x)-x$ 라는 잔차 함수를 학습한다. 그러면 출력은 $H(x)=F(x)+x$ 로 표현되며, 입력 x 를 지름길(shortcut)로 직접 더해 줄 수 있다. 이 구조는 역전파에서 그래디언트가 우회 경로를 통해 더 안정적으로 전달되게 하며, 깊은 네트워크에서도 학습 신호가 사라지지 않도록 도와준다.

- shortcut은 정보의 “우회 경로”를 제공해 깊이 증가에 따른 학습 난도를 낮춘다.
- 잔차 블록은 메인 경로가 학습에 실패해도 identity mapping이 최소 성능 기준선을 제공하게 만든다.
- ResNet의 강점은 단순히 층 수 증가가 아니라, 깊이를 안전하게 사용할 수 있게 한 구조적 설계에 있다.

기울기 소멸 문제와 레지듀얼 블록의 탄생

문제 (Problem): 네트워크 깊이와 성능의 역설

네트워크 깊이가 깊어진다고 무조건 성능이 향상되지 않음 (56층 모델이 20층 모델보다 성능 저하 발생).

해결책 (Solution): 레지듀얼 블록과 숏컷

기울기가 잘 전파될 수 있도록 숏컷(Shortcut, Skip Connection) 도입. 입력 x 를 활성화 함수 출력에 직접 더함.

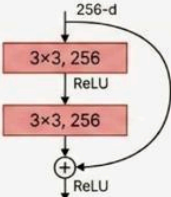
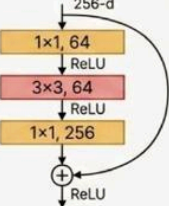
깊이 증가에 따른 성능 역설과 residual shortcut의 도입

3.1 기본 블록(Basic Block)과 병목 블록(Bottleneck)

ResNet18/34는 비교적 단순한 Basic Block을 사용한다. 기본 블록은 3×3 convolution 두 개와 shortcut 덧셈으로 이루어진다. 구조가 직관적이어서 이해하기 쉽고, 얇거나 중간 정도 깊이의 네트워크에 적합하다. 반면 ResNet50 이상은 Bottleneck Block을 사용한다. 이 블록은 $1\times1 \rightarrow 3\times3 \rightarrow 1\times1$ 구조로, 먼저 채널 수를 줄여 연산량을 아끼고, 가운데 3×3 convolution에서 실제 특징을 추출한 뒤, 마지막 1×1 convolution으로 채널 수를 다시 확장한다.

슬라이드에서는 Basic Block 기반 ResNet34와 Bottleneck 기반 ResNet50를 비교하면서, 더 깊은 네트워크임에도 병목 구조가 파라미터와 연산 효율 측면에서 더 유리할 수 있음을 강조한다. 즉, Bottleneck은 “더 깊되, 더 비싸지 않게” 만들기 위한 구조적 장치라고 이해할 수 있다.

- Basic Block: 3×3 conv 두 개 중심, 구현이 단순하고 직관적이다.
- Bottleneck: 1×1 conv로 차원 축소/복구를 수행해 깊은 모델에서 계산량을 줄인다.
- 1×1 convolution은 단순 보조 레이어가 아니라 채널 차원을 조절하는 핵심 제어 장치다.

아키텍처 확장: 기본 블록 vs 병목 블록		
	기본 블록 (ResNet18/34)	병목 블록 (ResNet50+)
구조 (Structure)	ResNet34 레지듀얼 블록 	ResNet50 레지듀얼 블록 
파라미터 수 (Parameters)	약 39.32M	약 6.96M (병목 블록이 깊이가 깊어져도 파라미터가 오히려 감소)
핵심 원리 (Key Mechanism)	1×1 합성곱으로 채널 수를 줄였다가 다시 늘려 차원 축소 및 연산량 최적화 수행.	

기본 블록과 병목 블록의 구조 및 파라미터 효율 비교

[코드 매핑] 병목 블록 (Bottleneck) 차원 조절 구현

```
self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=1,
                        stride=1, bias=False)
self.bn1 = nn.BatchNorm2d(out_channels)
self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3,
                        stride=stride, padding=1, bias=False)
self.bn2 = nn.BatchNorm2d(out_channels)
self.conv3 = nn.Conv2d(out_channels, self.expansion*out_channels,
                        kernel_size=1, stride=1, bias=False)
self.bn3 = nn.BatchNorm2d(self.expansion*out_channels)
self.relu = nn.ReLU(inplace=True)
```

차원 축소
(Dimension Reduction)

특성 추출
(Feature Extraction)

차원 복구 (Dimension Expansion)
- expansion*out_channels 적용

다양한 입력 데이터 처리 및 비선형성 증가를 위해 층을 깊게 쌓으면서도 연산 비용을 줄이는 설계.

병목 블록의 1×1-3×3-1×1 코드 매핑

3.2 PyTorch 코드 관점에서 본 residual block

PyTorch 구현에서 가장 중요한 줄은 입력을 임시로 저장한 뒤 마지막에 다시 더하는 부분이다. 보통 forward 내부에서 $i = x$ 또는 $identity = x$ 처럼 입력을 보존하고, 메인 경로의 convolution·batch normalization·ReLU를 통과한 결과와 더한다. 만약 공간 크기나 채널 수가 달라지는 구간이라면, shortcut 경로에도 1×1 convolution을 적용해 차원을 맞춘다. 이를 projection shortcut 또는 downsample shortcut이라고 부른다.

```
def forward(self, x):
    identity = x
    x = self.conv1(x)
    x = self.bn1(x)
    x = self.relu(x)
    x = self.conv2(x)
    x = self.bn2(x)

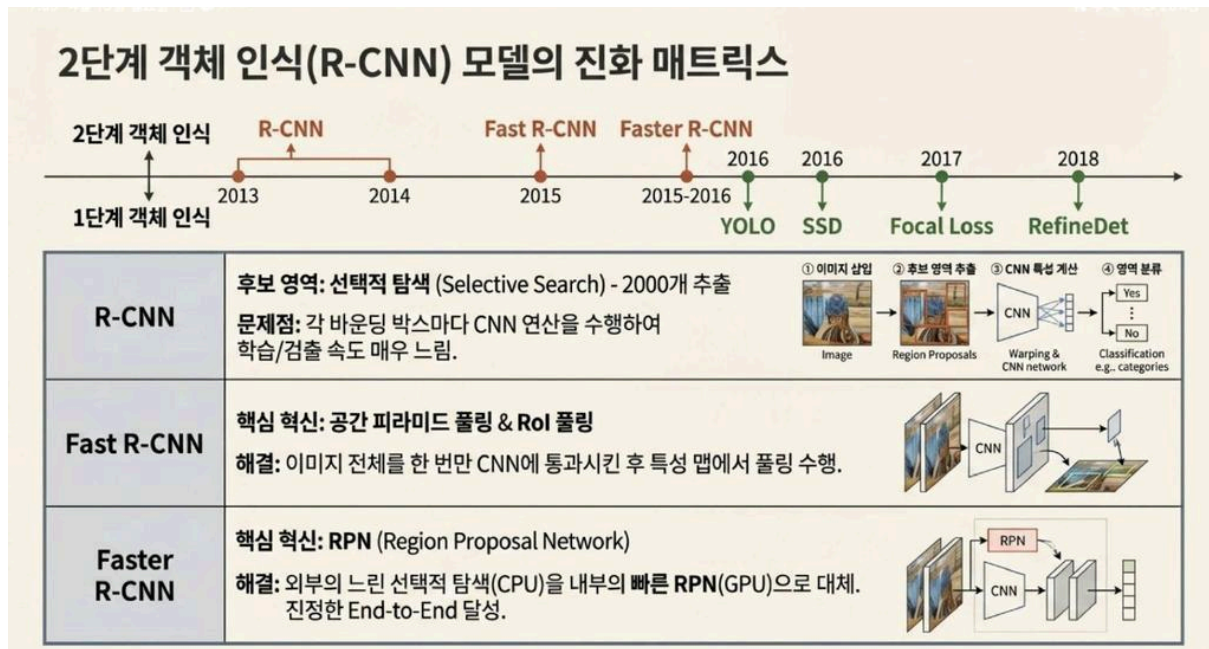
    if self.downsample is not None:
        identity = self.downsample(identity)

    x = x + identity
    x = self.relu(x)
    return x
```

위 코드는 residual learning의 핵심을 그대로 보여 준다. 메인 경로에서 새로운 특징을 학습하고, shortcut 경로는 기존 입력 신호를 보존해 더한다. 이 구조 덕분에 모델은 “새로 배워야 할 차이”에 집중할 수 있고, 깊이가 깊어져도 신호가 끊기지 않는다.

Fast R-CNN은 이미지를 한 번만 CNN에 통과시켜 feature map을 만들고, 후보 영역에 대해서는 RoI Pooling으로 고정 크기 특징을 추출하도록 바꾸었다. Faster R-CNN은 한 단계 더 나아가, 후보 영역 생성 자체를 외부 알고리즘이 아니라 네트워크 내부의 RPN이 맡도록 했다. 이로써 detection pipeline이 더 빠르고 일관된 end-to-end 구조가 되었다.

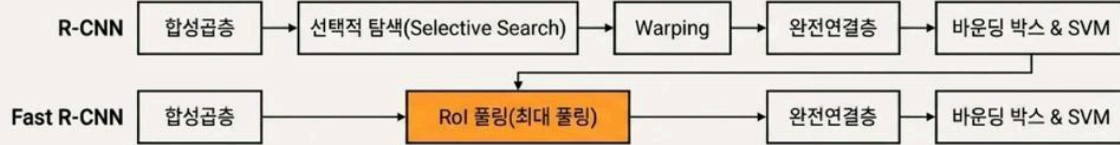
- R-CNN의 문제: 후보 영역마다 CNN을 반복 실행해야 하므로 느리다.
- Fast R-CNN의 핵심: convolution 연산 공유 + RoI Pooling으로 고정 크기 특징 생성.
- Faster R-CNN의 핵심: RPN으로 후보 영역 생성까지 내부화하여 탐지 병목을 해결.



2-stage detector의 진화 매트릭스

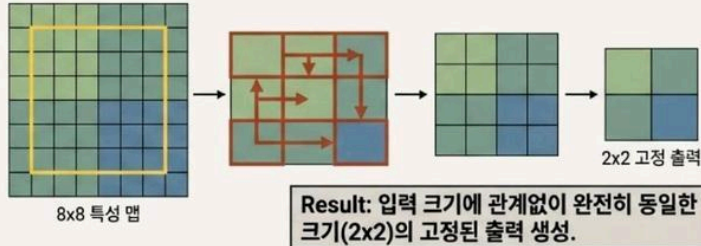
핵심 메커니즘: 공간 피라미드 풀링 & RoI 풀링

Fast R-CNN Architecture vs. R-CNN



RoI 풀링: 단계별 변환 과정 (8x8 → 2x2)

- Step 1: 8x8 특성 맵에 7x5 후보 영역 발생.
- Step 2: 출력 크기를 2x2로 맞추기 위해 스트라이드 계산 ($7/2=3$, $5/2=2$).
- Step 3: 가변적인 크기의 특성 맵 영역에 다르게 최대 풀링(Max Pooling) 적용.



R-CNN 대비 Fast R-CNN과 RoI Pooling의 동작 원리

병목 해결의 완성: Faster R-CNN과 RPN

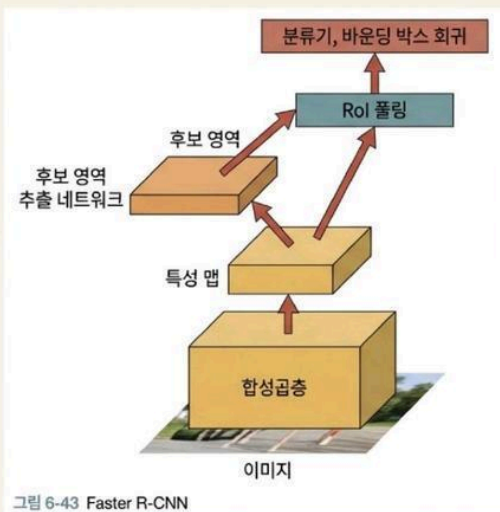
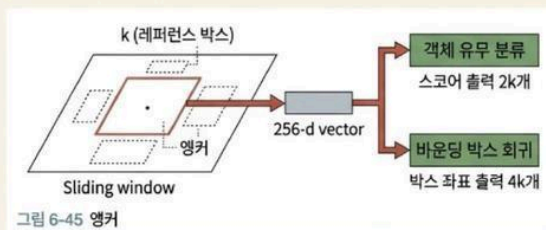


그림 6-43 Faster R-CNN



RPN (Region Proposal Network)

후보 영역 추출을 CNN 내부로 가져와 GPU 연산 활용.

앵커 박스 (Anchor Boxes): 다양한 객체의 크기와 비율을 포착하기 위해 미리 정의된 k개의 레퍼런스 박스 사용.

슬라이딩 윈도우 위치마다 객체 유무 분류(Score) 및 위치 조정(Bounding Box Regression) 동시 수행.

Faster R-CNN에서 RPN과 RoI Pooling의 결합 구조

4.1 RoI Pooling의 의미

RoI Pooling은 크기가 제각각인 후보 영역을 항상 같은 크기의 feature representation으로 바꾸는 레이어이다. 예를 들어 7x5 크기의 후보 영역이라도 2x2 혹은 7x7처럼 고정된 출력으로 변환할 수 있다.

이 덕분에 뒤쪽 **fully connected layer** 또는 **detection head**는 입력 크기에 상관없이 동일한 구조를 유지할 수 있다.

개념적으로는 후보 영역을 여러 개의 **bin**으로 나눈 뒤, 각 **bin**에서 **max pooling**을 수행해 대표값을 뽑는 방식이다. 이는 속도 면에서 강력했지만 좌표 양자화(**quantization**)에 따른 위치 오차를 유발할 수 있었고, 후속 연구에서는 **RoIAlign**이 이를 보완했다.

4.2 RPN과 anchor box

RPN(Region Proposal Network)은 **feature map** 위를 **sliding window** 방식으로 훑으면서, 각 위치에서 여러 개의 **anchor box**를 기준으로 객체 존재 확률(**objectness**)과 **bounding box** 보정값(**regression**)을 예측한다. **anchor**는 미리 정해진 다양한 크기와 종횡비의 기준 상자로, 한 번의 예측으로 여러 형태의 객체를 동시에 다룰 수 있게 한다.

중요한 점은 후보 제안이 이제 외부 **CPU** 알고리즘이 아니라 **GPU** 기반 **CNN** 내부에서 이뤄진다는 것이다. 이 변화는 속도 향상뿐 아니라 **feature sharing**, 공동 학습, 파이프라인 단순화라는 효과를 함께 가져왔다.

5. 분할의 출발점: FCN과 공간 정보 보존

분류 네트워크는 마지막 **fully connected layer**를 사용해 이미지를 하나의 벡터로 압축한다. 그러나 분할(**segmentation**)은 각 픽셀마다 예측해야 하므로, 이런 압축은 위치 정보를 잃게 만든다. **FCN**(Fully Convolutional Network)은 완전연결층을 **1×1 convolution**으로 대체하여, 입력의 공간 구조를 유지한 채 클래스별 점수 맵(**heatmap**)을 출력하도록 바꿨다.

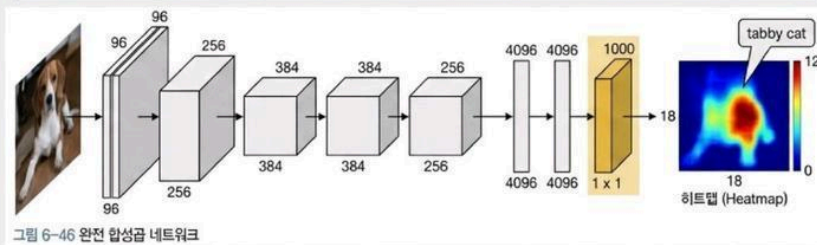
이 변화는 “이미지 한 장에 대한 하나의 클래스 예측”에서 “모든 위치에 대한 조밀한 예측(**dense prediction**)”으로 문제 정의를 바꾼 혁신이었다. 이후 등장한 **U-Net**, **DeepLab**, **PSPNet**은 모두 **FCN**이 연이 패러다임 위에서 발전했다고 볼 수 있다.

이미지 분할의 시작: 완전 합성곱 네트워크 (FCN)

문제 (Problem): 완전연결층(FC Layer)을
거치면 픽셀의 '위치 정보(Spatial Information)'가 사라짐.



해결책 (Solution): 완전연결층을 1x1 합성곱으로 대체하여 위치 정보 보존.



입력 이미지 크기에 대한 제약이 사라지고, 객체의 위치를 나타내는 히트맵(Heatmap) 도출 가능.

FCN: *fully connected layer*를 *1x1 convolution*으로 대체한 전환점

5.1 해상도 복구와 deconvolution

FCN 이후 분할 모델의 중요한 과제는 낮아진 **feature map** 해상도를 다시 복원하는 일이었다.

역합성곱(deconvolution 혹은 *transposed convolution*)은 주변에 0을 삽입하고 학습 가능한 필터를 적용하여 출력 크기를 키우는 방식으로 **upsampling**을 수행한다. 단순 보간보다 유연하지만,

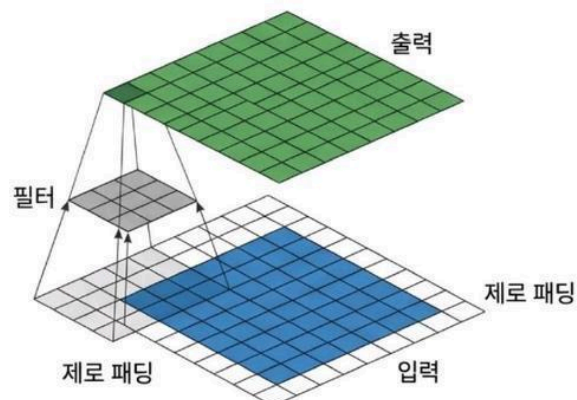
checkerboard artifact 같은 부작용이 생길 수 있으므로 실제 구현에서는 **bilinear upsampling + convolution** 조합도 많이 사용된다.

해상도 복구 메커니즘: 역합성곱 (Deconvolution)

합성곱 네트워크 (크기 축소/특성 추출) → 역합성곱 네트워크 (업샘플링/크기 복구)

Step 1: 원본 픽셀들 주위에 제로 패딩(Zero-padding) 추가.

Step 2: 패딩된 영역 위로 합성곱 연산을 수행하여 특성 맵의 크기를 물리적으로 증폭(Upsampling).

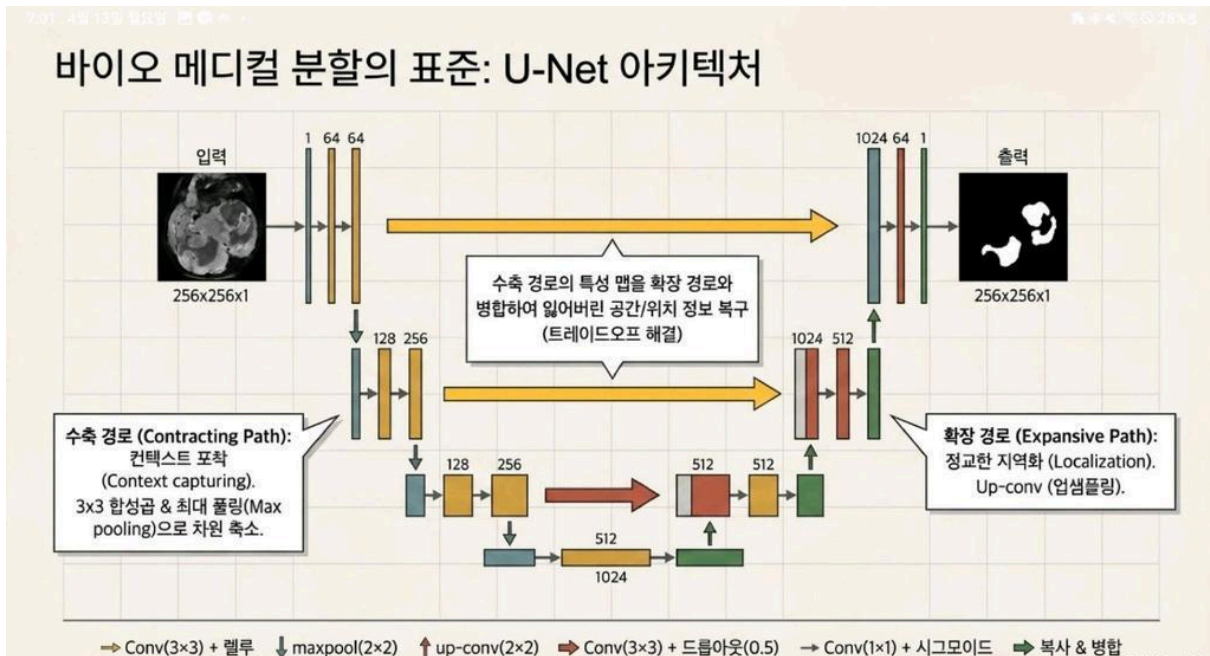


6. U-Net: 위치 복원을 위한 인코더-디코더 + 스킵 연결

U-Net은 특히 바이오메디컬 영상 분할에서 표준처럼 자리 잡은 구조다. 왼쪽의 contracting path는 convolution과 max pooling으로 문맥(context)을 축적하고, 오른쪽의 expansive path는 up-convolution으로 해상도를 복원한다. 여기서 핵심은 같은 해상도 단계의 encoder feature map을 decoder 쪽으로 복사해 붙이는 skip connection이다.

이 skip connection은 단순한 보조 연결이 아니다. 다운샘플링 과정에서 잃어버린 세부 경계와 위치 정보를 상위 수준 문맥과 결합해, “무엇이 있는가”와 “정확히 어디에 있는가”를 동시에 맞추게 한다. 그래서 U-Net은 의료 영상처럼 경계가 중요하고 데이터 수가 많지 않은 영역에서 특히 강력하다.

- 수축 경로: 의미적 문맥 확보
- 확장 경로: 해상도 복원 및 정교한 지역화
- 복사 & 병합: 위치 정보와 의미 정보를 결합하는 핵심 단계

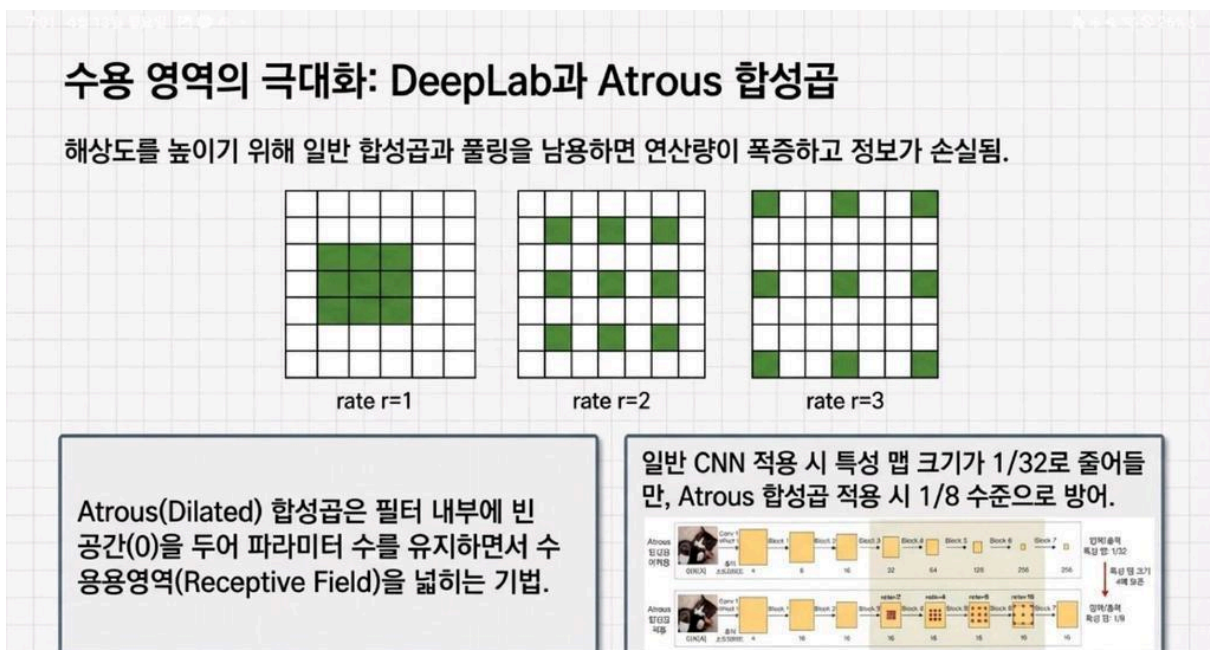


U-Net 구조: contracting path와 expansive path의 대칭적 결합

7. DeepLab: 넓은 문맥과 높은 해상도를 동시에 잡는 방법

DeepLab은 분할에서 중요한 두 요구를 동시에 잡으려 한다. 하나는 넓은 수용 영역(receptive field), 다른 하나는 픽셀 수준의 공간 해상도다. 일반 CNN처럼 stride와 pooling을 계속 사용하면 receptive field는 넓어지지만, 출력 해상도는 $1/32$ 수준으로 줄어 세부 경계가 무너진다. DeepLab은 atrous(dilated) convolution을 사용해 파라미터 수를 크게 늘리지 않으면서도 더 넓은 문맥을 볼 수 있게 한다.

Atrous convolution은 필터 원소 사이에 간격을 두어 sampling spacing을 확장한다. dilation rate가 커질수록 한 번에 보는 범위는 커지지만, 너무 크면 세밀한 구조를 놓칠 수 있다. 따라서 DeepLab은 적절한 rate 설계와 multi-scale 문맥 통합을 통해 해상도와 문맥 사이의 균형을 맞춘다.

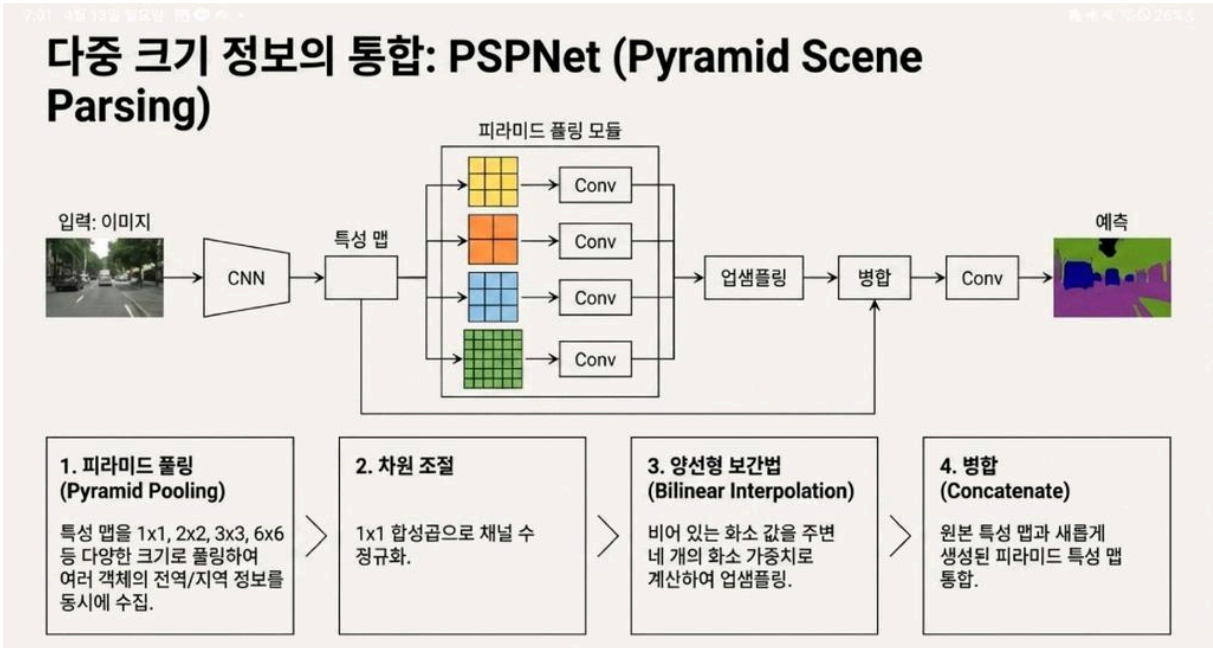


Atrous convolution의 dilation rate 변화와 수용 영역 확대

8. PSPNet: 전역 문맥과 지역 문맥의 동시 통합

PSPNet(Pyramid Scene Parsing Network)은 하나의 feature map을 여러 스케일의 bin으로 pooling하여 장면의 전역 정보와 지역 정보를 동시에 모으는 구조다. 1×1 , 2×2 , 3×3 , 6×6 등 다양한 크기의 피라미드 풀링 결과를 만든 뒤, 이를 1×1 convolution으로 차원 정렬하고 bilinear interpolation으로 원래 해상도에 맞춘 다음, 원본 feature map과 concatenate한다.

이 방식은 장면 파싱처럼 “큰 배경 맥락”과 “작은 객체 디테일”을 함께 알아야 하는 과제에서 유리하다. U-Net이 skip connection으로 저수준 공간 정보를 살린다면, PSPNet은 pyramid pooling으로 다중 스케일 문맥 자체를 강화한다는 점에서 서로 보완적이다.



PSPNet의 pyramid pooling과 feature fusion 흐름

9. 전체 정리: 왜 이 아키텍처들이 등장했는가

이 문서에서 살펴본 아키텍처들은 과제가 달라도 공통된 설계 철학을 공유한다. ResNet은 깊이 증가로 인한 최적화 실패를 shortcut으로 보완했고, Faster R-CNN은 검출 파이프라인의 중복 계산과 외부 병목을 feature sharing·RoI pooling·RPN으로 해결했으며, FCN/U-Net/DeepLab/PSPNet은 분할에서 사라지는 공간 정보를 1×1 convolution, upsampling, skip connection, atrous convolution, pyramid pooling으로 복구하거나 강화했다.

모델	해결하려는 병목	핵심 장치	핵심 효과
ResNet	깊은 네트워크의 기울기 소실/성능 저하	Residual block, identity shortcut	깊은 학습 안정화
Fast/Faster R-CNN	후보 영역 처리 속도와 다중 스케일 객체	RoI Pooling, RPN, anchor	검출 속도/정확도 개선
FCN	FC layer로 인한 공간 정보 손실	1×1 convolution	dense prediction 가능

U-Net	해상도 축소로 인한 경계 손실	Encoder-decoder, skip connection	정교한 위치 복원
DeepLab	문맥 확대와 해상도 유지의 충돌	Atrous convolution	넓은 receptive field + 높은 해상도
PSPNet	다중 스케일 문맥 부족	Pyramid pooling	전역/지역 정보 통합

실전 구현 관점에서는 두 가지가 중요하다. 첫째, PyTorch 코드를 읽을 때는 **tensor shape**가 어디서 바뀌는지, **skip connection**이나 **pooling**이 어떤 정보 손실/보존을 담당하는지 해석해야 한다. 둘째, torchvision의 사전학습 모델을 한 줄로 불러오더라도 내부 **block** 구조를 이해해야 **fine-tuning**, **head** 변경, **feature extraction**, **segmentation backbone** 재사용 같은 응용이 가능하다.